

spatial and temporal components within each transformer layer.

Both variants leverage the **ChessFormer** backbone [4] to extract rich spatial features from individual positions. Below, we detail the spatial feature extraction process and how temporal information is incorporated in each variant.

1) *Spatial Feature Extraction*: The spatial feature extraction consists of two main components: the ChessFormer backbone and the token transformation process. We employ the ChessFormer architecture as the backbone for our models due to its specialized design for processing chess positions.

Unlike traditional transformers that process tokens embedded in a topology whose structure roughly translates to Euclidean 1- and 2-space, ChessFormer addresses the unique topology of chess board, where the concept of distance is defined by chess-specific attributes rather than physical proximity. For example, squares that are a knight's, rook's or bishop's move away from each other are considered closely related even if they are far apart on the board. This is achieved by incorporating specialized positional encoding schemes within the self-attention mechanism, reflecting the inherent relationships between squares based on chess piece movement rather than Euclidean distance.

We utilize the variant of ChessFormer that introduces a learnable bias mechanism called *Smolgen* [16] for each square in the self-attention mechanism before the softmax function. This approach allows the model to capture the unique characteristics and importance of each square on the chessboard, effectively reflecting the spatial relationships inherent in chess.

The bias matrix $\mathbf{B}$ is critical in capturing the positional relationships between squares. For each block in the transformer encoder, it is obtained through a series of transformations applied to the token embeddings. Specifically, each token embedding $\mathbf{h}_i \in \mathbb{R}^d$, representing the state of square i , is first compressed to a lower-dimensional representation:

$$\mathbf{c}_i = \mathbf{h}_i \mathbf{W}_c, \quad \mathbf{W}_c \in \mathbb{R}^{d \times d_c},$$

where $d_c = d/32$ is the reduced dimension. The compressed tokens are then concatenated and reshaped into a single vector:

$$\mathbf{c} = \text{concat}(\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{64}) \in \mathbb{R}^{64 \cdot d_c}.$$

This vector is passed through a fully connected layer with activation function ϕ and layer normalization to obtain a hidden representation:

$$\mathbf{h} = \text{LayerNorm}(\phi(\mathbf{c} \mathbf{W}_h)), \quad \mathbf{W}_h \in \mathbb{R}^{(64 \cdot d_c) \times h_s},$$

where $h_s = d/4$ is the size of the hidden layer, and ϕ denotes the activation function.

Subsequently, the hidden representation $\mathbf{h}$ is transformed to generate bias embeddings for each attention head:

$$\mathbf{g} = \text{LayerNorm}(\phi(\mathbf{h} \mathbf{W}_g)), \quad \mathbf{W}_g \in \mathbb{R}^{h_s \times (h \cdot h_s)},$$

where h is the number of attention heads. The result $\mathbf{g} \in \mathbb{R}^{h \cdot h_s}$ is reshaped into $\mathbf{G} \in \mathbb{R}^{h \times h_s}$. A weight matrix $\mathbf{W}_b \in$

$\mathbb{R}^{h_s \times (64 \cdot 64)}$ shared between all encoder blocks is then used to compute the bias matrices for each attention head:

$$\mathbf{b}^{(j)} = \mathbf{G}^{(j)} \mathbf{W}_b, \quad \mathbf{b}^{(j)} \in \mathbb{R}^{64 \cdot 64},$$

where $\mathbf{G}^{(j)}$ is the bias embedding for head j . The bias vector $\mathbf{b}^{(j)}$ is reshaped into the bias matrix:

$$\mathbf{B}^{(j)} = \text{reshape}(\mathbf{b}^{(j)}, (64, 64)).$$

For each attention head j , the attention scores between tokens are computed as:

$$\mathbf{A}^{(j)} = \frac{\mathbf{Q}^{(j)} (\mathbf{K}^{(j)})^\top}{\sqrt{d_k}} + \mathbf{B}^{(j)},$$

where $\mathbf{Q}^{(j)}, \mathbf{K}^{(j)} \in \mathbb{R}^{64 \times d_k}$ are the query and key matrices for head j , and $d_k = d/h$ is the dimensionality of each head.

The significance of this positional encoding lies in its ability to model the intricate positional dependencies between squares on the chessboard. In chess, the influence of a piece is not determined by Euclidean distance but by complex movement patterns unique to each piece type. Standard positional encodings used in transformers are inadequate for capturing these relationships. By utilization of Smolgen mechanism as described, the ChessFormer backbone can effectively model these complex interactions.

Using the ChessFormer backbone, we obtain the token embeddings $\mathbf{H}_n \in \mathbb{R}^{64 \times d}$ for each board B_n . To obtain a hidden representation for each board, we perform a token transformation process in a similar manner to how ChessFormer obtains value embeddings. Each token $\mathbf{h}_{n,i}$ is linearly transformed to a lower-dimensional space:

$$\mathbf{z}_{n,i} = \mathbf{h}_{n,i} \mathbf{W}_1 + \mathbf{b}_1, \quad \mathbf{W}_1 \in \mathbb{R}^{d \times d_z}.$$

After transforming all tokens, we concatenate the resulting vectors:

$$\mathbf{z}_n = \text{concat}(\mathbf{z}_{n,1}, \dots, \mathbf{z}_{n,64}) \in \mathbb{R}^{64 \cdot d_z}.$$

Finally, we project this concatenated vector to a higher-dimensional space using the transformation:

$$\mathbf{s}_n = \mathbf{z}_n \mathbf{W}_2 + \mathbf{b}_2, \quad \mathbf{W}_2 \in \mathbb{R}^{(64 \cdot d_z) \times d_e}.$$

The resulting vector $\mathbf{s}_n \in \mathbb{R}^{d_e}$ serves as the embedding of board B_n for subsequent processing.

2) *Incorporating Temporal Information*: Before discussing our methods for incorporating temporal information, we must compare our approach to that of ChessFormer. ChessFormer integrates temporal dependencies by stacking several recent positions and embedding this temporal information directly into the tokens before processing them through the transformer encoder. In contrast, we process each position individually through the backbone and incorporate temporal dependencies at higher levels. This strategy allows us to capture complex temporal relationships.

We explore two methods for incorporating temporal information: the *Factorized Encoder Model* and the *Factorized Self-Attention Model*.

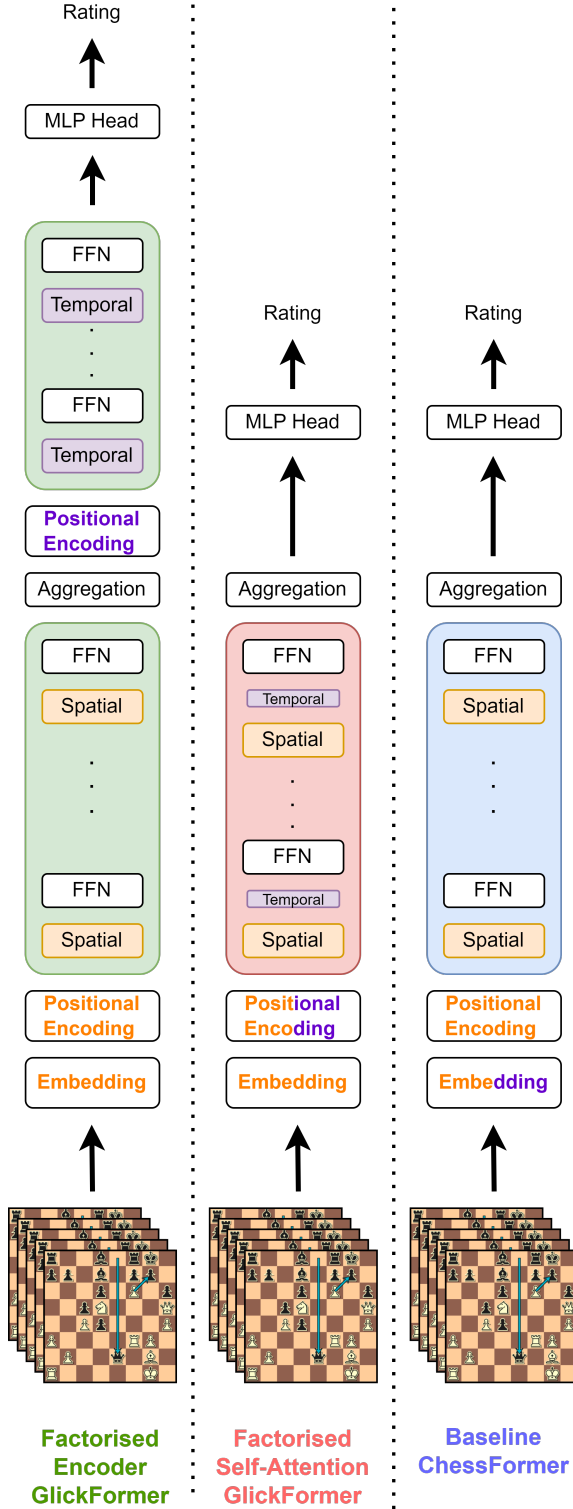


Fig. 3: We propose two different architectures that incorporate temporal information processing mechanisms on top of the baseline ChessFormer model. Additionally, we modify the positional embedding schemes to address the architectural modifications.

In the **Factorized Encoder Model**, we employ a late fusion strategy [17] to integrate spatial and temporal information. Initially, each board n is independently processed through ChessFormer encoder to obtain its representation s_n . This approach allows the model to capture intricate spatial features of each board without the influence of temporal context. We construct a sequence matrix of these spatial representations:

$$\mathbf{S} = [\mathbf{s}_1; \mathbf{s}_2; \dots; \mathbf{s}_N] \in \mathbb{R}^{N \times d_e}.$$

To incorporate temporal order and address the permutation invariance of self-attention mechanism, we add learnable positional encodings to each board representation:

$$\tilde{\mathbf{s}}_n = \mathbf{s}_n + \mathbf{t}_n, \quad \mathbf{t}_n \in \mathbb{R}^{d_e}.$$

Here, $\mathbf{t}_n$ is a learnable embedding that encodes the position n of each board in the sequence, ensuring that the model can distinguish between different time steps. We then process the sequence through L_t layers of transformer encoder blocks focusing on temporal relationships:

$$\mathbf{S}^{(l)} = \text{TransformerLayer}_{\text{Temporal}}^{(l)}(\mathbf{S}^{(l-1)}), \quad l = 1, \dots, L_t,$$

where $\mathbf{S}^{(0)} = [\tilde{\mathbf{s}}_1; \dots; \tilde{\mathbf{s}}_N]$. The representation corresponding to the initial position $\mathbf{s}_1^{(L_t)}$ is used to predict the puzzle's difficulty:

$$\hat{y} = f(\mathbf{s}_1^{(L_t)}),$$

with $f: \mathbb{R}^{d_e} \rightarrow \mathbb{R}$ being a Multilayer Perceptron.

In the **Factorized Self-Attention Model**, we integrate temporal attention directly within the ChessFormer blocks by adding temporal attention layers after spatial attention. Each encoder block consists of spatial attention with Smolgen as described in *paragraph 1*) of this subsection, temporal attention, and a feed-forward network (FFN). We apply residual connection and layer normalization after each of the aforementioned operation, which will be omitted from the following descriptions. Additionally we introduce a learnable embedding $\mathbf{t}_n$, which is added to each token embedding $\mathbf{H}_n$ to encode the temporal position of each board n in the sequence of boards. This temporal positional encoding resolves the self-attention invariance issue in temporal attention layers.

For each board n , we apply ChessFormer's multi-head self-attention to obtain updated token embeddings $\mathbf{H}_n^{(l,s)}$. We then stack the spatially attended tokens across all boards:

$$\mathbf{H}^{(l,s)} = [\mathbf{H}_1^{(l,s)}; \mathbf{H}_2^{(l,s)}; \dots; \mathbf{H}_N^{(l,s)}] \in \mathbb{R}^{N \times 64 \times d}.$$

This tensor is reshaped to group tokens by spatial position:

$$\mathbf{H}_{\text{reshaped}}^{(l,s)} \in \mathbb{R}^{64 \times N \times d}.$$

For each spatial position i , we extract the sequence of tokens across all boards:

$$\mathbf{H}_i^{(l,s)} = [\mathbf{h}_{1,i}^{(l,s)}; \mathbf{h}_{2,i}^{(l,s)}; \dots; \mathbf{h}_{N,i}^{(l,s)}] \in \mathbb{R}^{N \times d}.$$

We apply multi-head self-attention over the temporal dimension:

$$\mathbf{H}_i^{(l,t)} = \text{MultiHeadSelfAttention}(\mathbf{H}_i^{(l,s)}).$$

After temporal attention, we reconstruct and reshape the tensor back to its original dimensions:

$$\mathbf{H}_n^{(l,t)} = [\mathbf{h}_{n,1}^{(l,t)}; \mathbf{h}_{n,2}^{(l,t)}; \dots; \mathbf{h}_{n,64}^{(l,t)}] \in \mathbb{R}^{64 \times d}.$$

We then apply a position-wise FFN to each token:

$$\mathbf{H}_n^{(l)} = \text{FFN}(\mathbf{H}_n^{(l,t)}).$$

This process is repeated for L layers. After the final layer, we perform token transformation and aggregation to obtain the board representations $\mathbf{s}_n$. The representation corresponding to the initial board $\mathbf{s}_1$ is used to predict the puzzle’s difficulty:

$$\hat{y} = f(\mathbf{s}_1).$$

By integrating temporal information in these ways, GlickFormer effectively models the sequential nature of chess puzzles, capturing how the difficulty evolves with each move. The Factorized Encoder Model separates spatial and temporal processing, while the Factorized Self-Attention Model combines them within each layer, allowing us to explore different methods of capturing spatiotemporal dependencies.

C. Training Objective

Both models are trained to minimize the mean squared error between the predicted difficulty $\hat{y}$ and the ground truth difficulty y :

$$\mathcal{L} = \frac{1}{M} \sum_{i=1}^M (\hat{y}_i - y_i)^2,$$

where M is the number of training samples in batch.

To improve the model’s robustness and generalization, we incorporate uncertainty into the target labels by treating the standardized rating $\mu_i = \frac{r_i - 1516}{543}$ and its associated rating deviation $\phi_i = \frac{RD_i}{543}$ as parameters of a normal distribution. Specifically, during training, instead of using the standardized Glicko-2 rating μ_i directly as the target, we sample the target difficulty y_i from a normal distribution $\mathcal{N}(\mu_i, \phi_i^2)$, effectively simulating various plausible difficulty values for each puzzle.

This approach serves two purposes:

- **Data Augmentation:** By sampling from $\mathcal{N}(\mu_i, \phi_i^2)$, we introduce variation in the target values for each puzzle, generating a broader range of difficulty ratings. This acts as a form of augmentation by providing the model with slightly different training targets for the same input puzzle, encouraging the model to learn general features that perform well across a variety of possible difficulty levels.
- **Regularization:** The introduction of variability in the target difficulty serves as a natural form of regularization. The model is encouraged to predict difficulty ratings that are stable across different samplings of the target, preventing it from overfitting to a specific Elo value and helping it generalize better to unseen data. Additionally, by clipping the sampled values to the range $[\mu_i - 3\phi_i, \mu_i + 3\phi_i]$, we ensure that the sampled values remain within a reasonable range, preventing extreme outliers and further stabilizing training.

By combining these elements, the model is better equipped to handle the inherent uncertainty and variability in puzzle difficulty, resulting in improved performance and generalization.

D. Implementation Details

Following ChessFormer, we utilize the Mish activation function [18]

$$f(x) = x \tanh(\ln(1 + e^x))$$

across all of our models. This choice is driven by Mish’s smooth and non-monotonic properties, which have been shown to improve feature extraction and gradient flow in deep networks.

Key hyperparameters shared between all the models include:

- Embedding dimension $d = 256$.
- Reduced dimensions $d_c = d/32$ and $d_z = 32$ for token compression.
- Position embedding dimension $d_e = 2d = 512$.
- Number of encoder layers $L, L_t = 16$.
- Number of self-attention heads $h = 16$.
- Dimensions of linear projections in self-attention heads $d_k, d_q, d_v = d/h$.
- Maximum number of chessboard positions in the input sequence $N_{max} = 5$.

For linear projections d_k, d_q, d_v in the temporal self-attention heads of Factorized Self-Attention GlickFormer we use a reduced dimension of $d/2h$.

V. EXPERIMENTS

A. Experimental Setup

We conducted experiments to evaluate the performance of GlickFormer in predicting chess puzzle difficulty. The dataset was split into training and validation sets, with 4,158,000 chess puzzles used for training and 42,000 puzzles reserved for testing and validation. The test set size, representing approximately 1% of the total data, is sufficient to provide a reliable and statistically significant evaluation of the models’ performance due to its substantial size and diversity. All models were implemented using Tensorflow 2.13 and trained on Nvidia Quadro RTX 6000 GPUs.

We standardized the Glicko-2 ratings by subtracting the mean rating of 1516 and dividing by the standard deviation of 543 to facilitate training. During training, we sampled target ratings from a normal distribution centered at the standardized rating with a variance based on the rating deviation, as described in subsection IV-C.

The models were trained for 28,000 steps using the RMSprop optimizer with a learning rate of 1×10^{-6} , $\rho = 0.99$ and a batch size of 4096. We observed that momentum-based optimizers like Adam or higher learning rates caused the models to converge prematurely to predicting the mean of the dataset with zero standard deviation. Additionally, we employed a cyclical optimizer state restarting strategy [19], where the length of the k -th cycle was set to $1000k$ steps.